Implementación del medidor de ritmo cardíaco

Circuitos Eléctricos II y Laboratorio

Material de referencia para reproducir el montaje y configurar el programa. Contiene las conexiones de acondicionamiento, los valores de los componentes, los cálculos de ganancias y filtros, y los dos programas del proyecto.

Fuente: Jorge Barrera y Juan Manuel Restrepo, Proyecto final Medidor de ritmo cardiaco, semestre 2026-1. Circuito y cálculos: páginas 3 a 5. Código: páginas 12 a 20. Se conservan los valores del trabajo y se precisan los resultados numéricos.

1 Circuito y valores de referencia

Etapa	Componentes y conexiones
Sensor óptico	LED rojo con resistencia serie de 100 Ω y receptor óptico. La conexión y polaridad del receptor deben tomarse de la Figura 1 de la página 3 del trabajo base. Confirmar la referencia del receptor y la tensión de alimentación antes del montaje.
Acoplamiento y buffer	Salida del sensor → condensador serie de 1 µF → nodo con resistencia de 100 kΩ a tierra → entrada no inversora del buffer. Unir salida y entrada inversora del buffer.
Preamplificación	Amplificador no inversor. Rf = 33 kΩ entre salida y entrada inversora. Rg = 1,8 kΩ entre entrada inversora y tierra. Señal del buffer a la entrada no inversora.
Sallen Key	R1 = R2 = 68 kΩ y C1 = C2 = 1 µF. Operacional como seguidor. Usar las conexiones de la figura de la página siguiente.
Amplificación final	Amplificador no inversor. Rf = 68 kΩ entre salida y entrada inversora. Rg = 2,2 kΩ entre entrada inversora y tierra. Señal filtrada a entrada no inversora.
Entrada del ESP32	Salida final → resistencia serie de 1 kΩ → ADC GPIO1. Zener de 3,3 V entre el nodo del ADC y tierra, con cátodo al nodo y ánodo a tierra. Masa común entre circuito y microcontrolador.

Operacionales: LM324. Microcontrolador indicado: ESP32 C3 Super Mini. La lista de materiales también debe incluir los elementos de alimentación y desacoplamiento del montaje.

El circuito de referencia incluye una resistencia y un Zener para la entrada del ADC. Esta protección no corresponde a una red snubber.

2 Conexiones del filtro y de los amplificadores

Filtro Sallen Key con ganancia unitaria. Dibujo de conexión de la topología estándar indicada en el trabajo. Verificar su correspondencia con la Figura 2 del original antes de cablear.

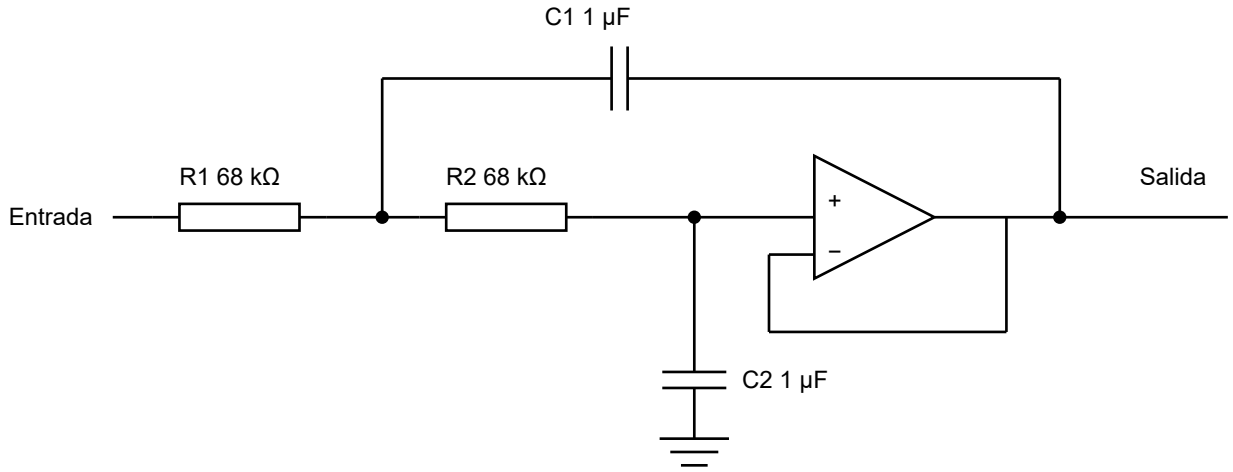


Figura 1. Conexiones del filtro Sallen Key de segundo orden con seguidor de tensión.

Configuración	Conexión
Buffer	Entrada a (+). Salida unida a (−). Ganancia unitaria.
No inversor	Entrada a (+). Rf desde salida a (−). Rg desde (−) a tierra. Usar los pares de resistencias de la sección 1.
Orden de montaje	Sensor → pasaaltas → buffer → preamplificador → Sallen Key → amplificador final → protección → ADC.

Verificación del montaje: confirmar el pinout del encapsulado LM324, la polaridad de los condensadores si son electrolíticos y las tensiones de alimentación. El texto del trabajo no especifica de forma suficiente la polarización DC de la señal ni todas las conexiones del sensor. Estas deben verificarse en el esquema original; no se debe asumir que una señal bipolar puede aplicarse directamente al ADC.

Esquema original para consultar la etapa óptica: Proyecto final, Figura 1, página 3. El acceso al enlace depende de los permisos de quien lo abre.

3 Cálculos de ganancias y filtros

Para las dos etapas no inversoras se utiliza A = 1 + Rf/Rg. Las siguientes ganancias corresponden al modelo ideal de cada amplificador.

Etapa	Sustitución	Resultado
Buffer	A = 1	1 V/V
Preamplificación	Apre = 1 + 33 / 1,8	19,333 V/V
Filtro con seguidor	K = 1	1 V/V en DC
Amplificación final	Af = 1 + 68 / 2,2	31,909 V/V
Producto de ganancias	Apre × Af	616,91 V/V ≈ 55,80 dB

El trabajo redondea las ganancias a 19 y 32. El producto 616,91 corresponde a las etapas amplificadoras; la ganancia desde el sensor hasta el ADC depende además de la frecuencia por la presencia de los filtros.

Filtro pasaaltas

$f_{HP} = 1 / (2\pi RC) = 1 / (2\pi \times 100\,000 \times 1 \times 10^{-6}) = 1{,}592$ Hz

Este cálculo se añade a partir de los componentes especificados en el trabajo.

Filtro Sallen Key

$f_0 = 1 / [2\pi \sqrt{(R1\ R2\ C1\ C2)}]$

$f_0 = 1 / (2\pi \times 68\,000 \times 1 \times 10^{-6}) = 2{,}341$ Hz

Precisión de diseño: 2,341 Hz es la frecuencia natural. Para la topología estándar de la Figura 1, con resistencias y condensadores iguales y K = 1, Q = 1/(3 − K) = 0,5. Por tanto, la frecuencia de −3 dB del pasabajas aislado es:

$f_{-3dB} = f_0 \sqrt{(\sqrt{2} - 1)} \approx 1{,}506$ Hz

El trabajo llama frecuencia de corte a 2,34 Hz. Antes de reproducir esa afirmación, comprobar la respuesta del filtro en simulación. La frecuencia de −3 dB del conjunto pasaaltas y pasabajas debe obtenerse de su respuesta completa.

Conversión de frecuencia a pulsaciones

$BPM = 60\ f = 60 / T = 60\,000 / \Delta t_{ms}$

Ejemplo del trabajo: f = 1,31 Hz → BPM = 78,6.

4 Parámetros de implementación

Parámetro del código	Valor de referencia
Entrada analógica	sensorPin = 1
Umbral superior e inferior	1,95 V y 1,83 V
Intervalos aceptados	450 ms < Δt < 1400 ms; aproximadamente 42,9 a 133,3 BPM
Suavizado	BPM nuevo = 0,7 × BPM previo + 0,3 × BPM instantáneo
Intervalo de lectura nominal	50 µs; 20 000 lecturas/s antes del promedio
Promedio por dato transmitido	50 lecturas → 2,5 ms por dato → 400 datos/s nominales
Tamaño del lote	250 datos → 0,625 s por lote nominal
Ventana de la interfaz	2500 datos → 6,25 s a 400 datos/s

Los tiempos son nominales: analogRead, la planificación de tareas y las comunicaciones pueden modificar el intervalo real. Medirlo durante la implementación.

Preparación de los programas

1. Crear un proyecto Arduino para la placa ESP32 C3 utilizada y colocar el código del anexo A en el archivo principal. Ajustar NOMBRE_RED, CLAVE_RED y serverName. El último debe contener la dirección IPv4 del computador que ejecuta el servidor y el puerto 5000.

2. Guardar el código del anexo B como servidor.py. En el computador instalar Flask con `python -m pip install flask` y ejecutar `python servidor.py`. Conectar computador y ESP32 a la misma red y permitir el acceso local al puerto configurado.

3. Abrir `http://IP_DEL_COMPUTADOR:5000/` en el navegador. Chart.js se obtiene de una CDN y requiere acceso a Internet para cargar la gráfica.

Ajustes que deben verificarse

La interfaz original indica 1 kHz y 500 ms/div; esos rótulos no coinciden con los 400 datos/s nominales. Ajustar la escala con el intervalo medido. El doble buffer del programa carece de un mecanismo de exclusión o cola: si el envío HTTP se retrasa, pueden sobrescribirse datos. El cálculo ADC × 3,3 / 4095 es una aproximación y debe contrastarse con una medición de voltaje.

El anexo A reproduce el código del trabajo, con credenciales sustituidas. Incluye una instrucción que deshabilita la detección de baja alimentación; revisar esa línea y las cabeceras soc para la placa y versión del entorno utilizadas. No se ha compilado ni probado en hardware. El anexo B conserva la interfaz original; se verificó su sintaxis Python.

Antes de conectar el ADC, medir su tensión y confirmar que permanece dentro del rango admitido por la placa. Ajustar ganancias, referencia DC y protección según la señal real.

Anexo A Código del ESP32

```
#include <Arduino.h>
#include <WiFi.h>
#include <HTTPClient.h>
#include "soc/soc.h"
#include "soc/rtc_cntl_reg.h"

void conectarWiFi();
void TareaWiFi(void * pvParameters);

const char* ssid = "NOMBRE_RED";
const char* password = "CLAVE_RED";
const char* serverName = "http://192.168.0.100:5000/data";

const int sensorPin = 1;

float threshold = 1.95;
bool beatDetected = false;
unsigned long lastBeat = 0;
int bpm = 0;


const unsigned long sampleInterval = 50;

unsigned long lastSampleTime = 0;

long sumaMicroLecturas = 0;
int contadorMicroLecturas = 0;

const int RATIO_COMPRESION = 50;


const int BATCH_SIZE = 250;

float bufferVoltaje[2][BATCH_SIZE];

volatile int indexEscritura = 0;

volatile int bufferActivoEscritura = 0;

volatile int bufferListoParaEnviar = -1;
WiFiClient client;
HTTPClient http;

void conectarWiFi() {

  WiFi.mode(WIFI_STA);

  WiFi.begin(ssid, password);

  while (WiFi.status() != WL_CONNECTED) {
    vTaskDelay(pdMS_TO_TICKS(300));
  }

  http.setReuse(true);
}


void TareaWiFi(void * pvParameters) {

  conectarWiFi();

  for(;;) {

```

Anexo A Código del ESP32 continuación

```
    if (bufferListoParaEnviar != -1 &&
        WiFi.status() == WL_CONNECTED) {

      int bufferAEnviar = bufferListoParaEnviar;

      http.begin(client, serverName);

      http.addHeader("Content-Type", "application/json");

      String json;

      json.reserve(BATCH_SIZE * 8 + 30);

      json = "{\"v\":[";

      for(int i = 0; i < BATCH_SIZE; i++) {

        json += String(bufferVoltaje[bufferAEnviar][i], 4);

        if(i < BATCH_SIZE - 1) {
          json += ",";
        }
      }

      json += "],\"b\":";

      json += String(bpm);

      json += "}";

      http.POST(json);

      http.end();

      bufferListoParaEnviar = -1;
    }

    vTaskDelay(pdMS_TO_TICKS(10));
  }
}

void setup() {

  WRITE_PERI_REG(RTC_CNTL_BROWN_OUT_REG, 0);

  Serial.begin(115200);

  analogReadResolution(12);

  analogSetAttenuation(ADC_11db);

  xTaskCreatePinnedToCore(
       TareaWiFi,
       "TaskWiFi",
       10000,
       NULL,
       1,
       NULL,
       0
  );
}

void loop() {
```

Anexo A Código del ESP32 continuación

```
127
128    unsigned long currentMicros = micros();
129
130
131    if (currentMicros - lastSampleTime >= sampleInterval) {
132
133       lastSampleTime = currentMicros;
134
135
136       int lectura = analogRead(sensorPin);
137
138       sumaMicroLecturas += lectura;
139
140       contadorMicroLecturas++;
141
142
143       if (contadorMicroLecturas >= RATIO_COMPRESION) {
144
145         float adc = sumaMicroLecturas / (float)RATIO_COMPRESION;
146
147         float voltaje = (adc * 3.3) / 4095.0;
148
149         sumaMicroLecturas = 0;
150
151         contadorMicroLecturas = 0;
152
153
154         unsigned long currentMillis = millis();
155
156         float thresholdHigh = threshold;
157
158            float thresholdLow = threshold - 0.12;
159
160            // Detectar flanco subida
161            if (voltaje > thresholdHigh && !beatDetected) {
162
163                beatDetected = true;
164
165                unsigned long intervaloPulso =
166                    currentMillis - lastBeat;
167
168                // FILTRO LATIDOS VÁLIDOS
169                if (intervaloPulso > 450 &&
170                    intervaloPulso < 1400) {
171
172                    int bpmInstantaneo =
173                        60000 / intervaloPulso;
174
175                    // SUAVIZADO
176                    bpm = (bpm * 0.7) +
177                          (bpmInstantaneo * 0.3);
178
179                    Serial.print("BPM: ");
180                    Serial.println(bpm);
181                }
182
183                lastBeat = currentMillis;
184            }
185
186            // Reinicio histéresis
187            if (voltaje < thresholdLow) {
188              beatDetected = false;
189            }
```

Anexo A Código del ESP32 continuación

```
190
191
192           if (indexEscritura < BATCH_SIZE) {
193
194               bufferVoltaje
195                 [bufferActivoEscritura]
196                 [indexEscritura] = voltaje;
197
198               indexEscritura++;
199           }
200
201           if (indexEscritura >= BATCH_SIZE) {
202
203               bufferListoParaEnviar =
204                   bufferActivoEscritura;
205
206               // Cambiar buffer
207               bufferActivoEscritura =
208                   (bufferActivoEscritura == 0) ? 1 : 0;
209
210               indexEscritura = 0;
211           }
212       }
213   }
214 }
```

Anexo B Código del servidor y la interfaz

```
from flask import Flask, request, jsonify
from collections import deque

app = Flask(__name__)

# 2500 datos = 2.5 segundos exactos a 1kHz
MAX_DATOS = 2500
voltajes = deque(maxlen=MAX_DATOS)
ultimo_bpm = 0

HTML = """
<!DOCTYPE html>
<html>
<head>
<title>Osciloscopio ESP32 - 1 kHz</title>
<script src="https://cdn.jsdelivr.net/npm/chart.js"></script>

<style>
body{
    background:#101010;
    color:white;
    font-family:Arial;
    text-align:center;
    margin:0;
}

.container{
    width:950px;
    margin:auto;
    margin-top:20px;
}

.top{
    display:flex;
    justify-content:center;
    gap:20px;
    margin-bottom:20px;
}

.box{
    width:180px;
    background:#1e1e1e;
    padding:15px;
    border-radius:15px;
    transition:0.3s;
}

.valor{
    font-size:40px;
    margin-top:10px;
    font-weight:bold;
}

canvas{
    background:black;
    border-radius:10px;
    border: 2px solid #333;
}
</style>
</head>

<body>

```

Anexo B Código del servidor y la interfaz continuación

```
<div class="container">
    <h1>Osciloscopio ESP32 (500ms / div)</h1>

    <div class="top">

         <div class="box" id="bpm_box">
             <div>BPM</div>
             <div class="valor" id="bpm">0</div>
         </div>

         <div class="box">
             <div>Voltaje</div>
             <div class="valor" id="voltaje">0</div>
         </div>

    </div>

    <canvas id="chart" width="950" height="350"></canvas>
</div>

<script>

const MAX_DATOS = 2500;

let labels_grid = [];
for(let i=0; i<MAX_DATOS; i++){
    labels_grid.push(i);
}

const ctx = document.getElementById('chart');

const chart = new Chart(ctx, {
    type:'line',
    data:{
         labels: labels_grid,
         datasets:[{
             data: new Array(MAX_DATOS).fill(null),
             borderWidth: 1.5,
             pointRadius: 0,
             tension: 0,
             borderColor: '#00ff00',
             spanGaps: true
         }]
    },
    options:{
         responsive:false,
         animation:false,
         plugins:{
             legend:{ display:false }
         },
         scales:{
             x:{
                  display:true,
                  title:{
                      display:true,
                      text:'Tiempo (500ms / div)'
                  },
                  grid:{
                      color:'rgba(255,255,255,0.2)'
                  },
                  ticks:{
                      callback: function(value, index){
                           return index % 500 === 0 ? '' : null;
```

Anexo B Código del servidor y la interfaz continuación

```
                        },
                        maxRotation:0
                    }
                },
                y:{
                    min:0.0,
                    max:3.0,
                    grid:{
                        color:'rgba(255,255,255,0.2)'
                    },
                    title:{
                        display:true,
                        text:'Voltaje'
                    }
                }
            }
    }
});

async function actualizar(){

    try{

         const response = await fetch('/get_data');
         const data = await response.json();

         const bpm = data.bpm;

         document.getElementById("bpm").innerHTML = bpm;

         // CAMBIO DE COLOR SEGÚN BPM
         const bpmBox = document.getElementById("bpm_box");

         if(bpm >= 60 && bpm <= 69){
             bpmBox.style.background = "#00aa00"; // VERDE
         }
         else if(bpm >= 70 && bpm <= 85){
             bpmBox.style.background = "#d4b000"; // AMARILLO
         }
        else if(bpm >= 86){
             bpmBox.style.background = "#cc0000"; // ROJO
        }
        else{
             bpmBox.style.background = "#1e1e1e"; // NORMAL
        }

        if(data.valores.length > 0){
             document.getElementById("voltaje").innerHTML =
                 data.valores[data.valores.length - 1].toFixed(2);
        }

        let valores_grafica = [...data.valores];

        while(valores_grafica.length < MAX_DATOS){
             valores_grafica.unshift(null);
        }

        chart.data.datasets[0].data = valores_grafica;

        chart.update('none');

    }
    catch(e){
```

Anexo B Código del servidor y la interfaz continuación

```
        console.log("Error:", e);
    }
    finally{
        setTimeout(actualizar, 40);
    }
}

actualizar();

</script>

</body>
</html>
"""

@app.route('/')
def home():
    return HTML

@app.route('/data', methods=['POST'])
def data():
    global ultimo_bpm

    contenido = request.json

    for v in contenido["v"]:
        voltajes.append(v)

    ultimo_bpm = contenido["b"]

    return jsonify({"ok":True})

@app.route('/get_data')
def get_data():
    return jsonify({
        "valores": list(voltajes),
        "bpm": ultimo_bpm
    })

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000, threaded=True)
```